多线程

多线程
例程实验效果简介
代码讲解
多线程模块 (_thread)
类型
`LockType`
方法:
函数
`allocate_lock()`
`exit()`
`get_ident()`
`stack_size([size])`
`start_new_thread(function, args)`

例程实验效果简介

本节例程代码位于：【源码汇总 / 02.Basic / 13.thread.py】

我们用CanMV IDE打开例程代码，将K230用usb连接到电脑上

点击CanMV IDE左下角的运行按钮，同时打开下面的串行终端

可以看到串行终端里面会交替循环输出

This is thread THREAD_1
This is thread THREAD_2



代码讲解

本节主要用到了多线程 _thread模块，代码如下：

```
# 导入线程模块 Import thread module
import _thread
# 导入时间模块用于实现延时 Import time module for delay functionality
import time
```

```
# 定义线程执行的函数 Define the function to be executed in threads
# name: 线程名称参数 Thread name parameter
def func(name):
    while True:
        # 每隔一秒输出一次信息 Print message every second
        print("This is thread {}".format(name))
        # 休眠1秒 Sleep for 1 second
        time.sleep(1)

# 创建并启动第一个线程 Create and start the first thread
# func: 线程函数 Thread function
# ("THREAD_1",): 传递给线程函数的参数(必须是元组格式)
# Arguments passed to thread function (must be tuple format)
_thread.start_new_thread(func,("THREAD_1",))

# 延时500毫秒
# Delay 500ms to give the first thread a chance to start
time.sleep_ms(500)

# 创建并启动第二个线程 Create and start the second thread
# 参数与第一个线程类似 Similar parameters as the first thread
_thread.start_new_thread(func,("THREAD_2",))

# 主线程死循环,防止程序退出
# Main thread infinite loop to prevent program exit
# 延时1毫秒,避免占用过多CPU资源
# Delay 1ms to avoid consuming too much CPU
while True:
    time.sleep_ms(1)
```

这个程序是一个多线程示例，展示了如何在MicroPython中创建和运行并发线程。主要功能如下：

1. 程序创建了两个并发运行的线程，分别命名为"THREAD_1"和"THREAD_2"
2. 每个线程执行相同的函数(func)，这个函数会：
    - 循环执行
    - 打印当前线程的名称
    - 每次打印后暂停1秒
3. 程序的执行顺序是：
    - 首先启动第一个线程
    - 暂停500毫秒（让第一个线程有时间启动）
    - 然后启动第二个线程
    - 最后主线程进入死循环（保持程序运行）
4. 运行效果：
    - 两个线程会交替打印各自的信息
    - 每个线程大约每秒钟打印一次
    - 由于线程是并发的，所以两个线程的输出会交织在一起

多线程模块 (_thread)

_thread是MicroPython的一个基本线程模块,用于实现多线程编程

这里有几点是在使用过程中需要着重注意的

> 1. _thread 模块的多线程实现的是系统级别的多线程
>
>   这意味着程序从硬件的角度上来看其实还是单线程进行的，只是Micropython通过内部的线程调度机制
>
>   模拟的实现多线程的效果。
> 2. _thread 模块的线程调度机制是非抢占式的
>
>   这意味着你必须**手动的**去避免某一个线程一直占用处理器资源，通常的做法是在每个线程（如果有循环的话）
>
>   在循环的结尾处添加一个主动的延迟函数time.sleep_us(1)。只有当执行到sleep的时候，系统才会进行一次线程的调度
> 3. python的_thread模块经过了多次的更新，在3.8以前都未被正式的确定使用，
>
>   K230固件中的Micropython对_thread的使用方法并不一定完全符合最新版python中的文档

类型

`LockType`

线程锁类型，用于线程同步。

方法:

- `acquire()`: 获取锁。如果锁已被其他线程持有，将阻塞直到锁被释放。
- `locked()`: 返回锁的状态。如果锁被某个线程持有返回 `True`，否则返回 `False`。
- `release()`: 释放锁。只能由持有锁的线程调用。

函数

`allocate_lock()`

创建并返回一个新的锁对象。

`exit()`

终止调用它的线程。

`get_ident()`

返回当前线程的标识符。

`stack_size([size])`

设置或获取新创建线程的栈大小。

`start_new_thread(function, args)`

启动一个新线程，执行给定的函数，并传入指定的参数元组。